

VOYAGER: An Open-Ended Embodied Agent with Large Language Models

Guanzhi Wang^{1,2}✉, Yuqi Xie³, Yunfan Jiang^{4*}, Ajay Mandlekar^{1*},
Chaowei Xiao^{1,5}, Yuke Zhu^{1,3}, Linxi “Jim” Fan^{1†}✉, Anima Anandkumar^{1,2†}

¹NVIDIA, ²Caltech, ³UT Austin, ⁴Stanford, ⁵UW Madison

*Equal contribution †Equal advising ✉ Corresponding authors

<https://voyager.minedojo.org>

Abstract

We introduce VOYAGER, the first LLM-powered embodied lifelong learning agent in Minecraft that continuously explores the world, acquires diverse skills, and makes novel discoveries without human intervention. VOYAGER consists of three key components: 1) an automatic curriculum that maximizes exploration, 2) an ever-growing skill library of executable code for storing and retrieving complex behaviors, and 3) a new iterative prompting mechanism that incorporates environment feedback, execution errors, and self-verification for program improvement. VOYAGER interacts with GPT-4 via blackbox queries, which bypasses the need for model parameter fine-tuning. The skills developed by VOYAGER are temporally extended, interpretable, and compositional, which compounds the agent’s abilities rapidly and alleviates catastrophic forgetting. Empirically, VOYAGER shows strong in-context lifelong learning capability and exhibits exceptional proficiency in playing Minecraft. It obtains $3.3\times$ more unique items, travels $2.3\times$ longer distances, and unlocks key tech tree milestones up to $15.3\times$ faster than prior SOTA. VOYAGER is able to utilize the learned skill library in a new Minecraft world to solve novel tasks from scratch, while other techniques struggle to generalize.

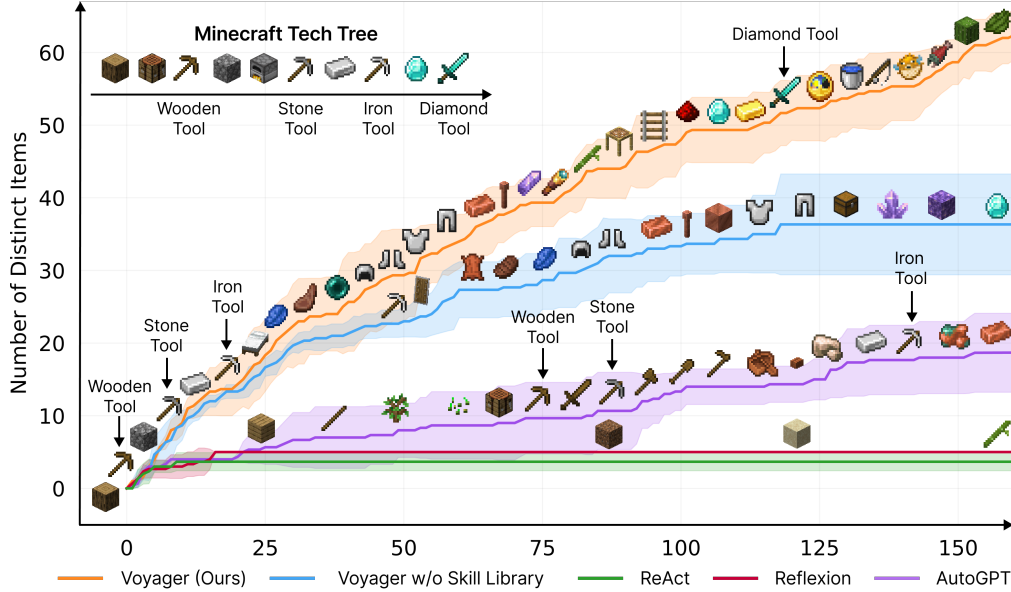


Figure 1: VOYAGER discovers new Minecraft items and skills continually by self-driven exploration, significantly outperforming the baselines. X-axis denotes the number of prompting iterations.

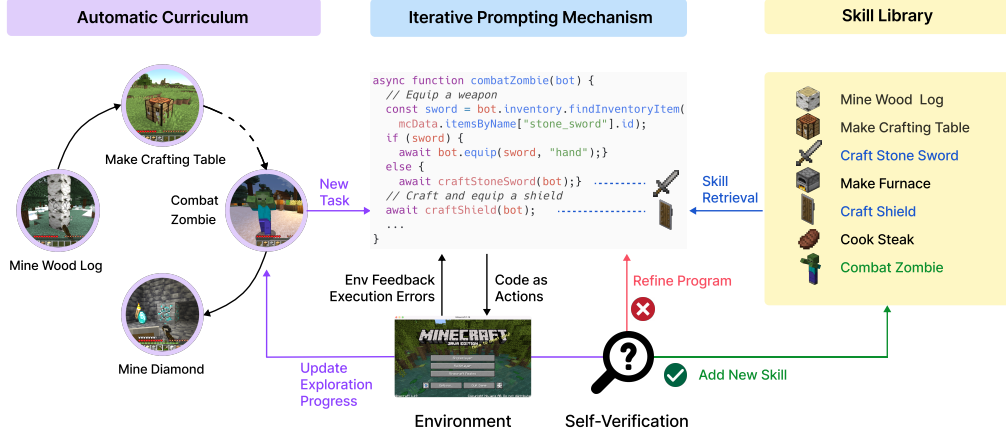


Figure 2: VOYAGER consists of three key components: an automatic curriculum for open-ended exploration, a skill library for increasingly complex behaviors, and an iterative prompting mechanism that uses code as action space.

1 Introduction

Building generally capable embodied agents that continuously explore, plan, and develop new skills in open-ended worlds is a grand challenge for the AI community [1–5]. Classical approaches employ reinforcement learning (RL) [6, 7] and imitation learning [8–10] that operate on primitive actions, which could be challenging for systematic exploration [11–15], interpretability [16–18], and generalization [19–21]. Recent advances in large language model (LLM) based agents harness the world knowledge encapsulated in pre-trained LLMs to generate consistent action plans or executable policies [16, 22, 19]. They are applied to embodied tasks like games and robotics [23–27], as well as NLP tasks without embodiment [28–30]. However, these agents are not lifelong learners that can progressively acquire, update, accumulate, and transfer knowledge over extended time spans [31, 32].

Let us consider Minecraft as an example. Unlike most other games studied in AI [33, 34, 10], Minecraft does not impose a predefined end goal or a fixed storyline but rather provides a unique playground with endless possibilities [23]. Minecraft requires players to explore vast, procedurally generated 3D terrains and unlock a tech tree using gathered resources. Human players typically start by learning the basics, such as mining wood and cooking food, before advancing to more complex tasks like combating monsters and crafting diamond tools. We argue that an effective lifelong learning agent should have similar capabilities as human players: (1) **propose suitable tasks** based on its current skill level and world state, e.g., learn to harvest sand and cactus before iron if it finds itself in a desert rather than a forest; (2) **refine skills** based on environmental feedback and **commit mastered skills to memory** for future reuse in similar situations (e.g. fighting zombies is similar to fighting spiders); (3) **continually explore the world** and seek out new tasks in a self-driven manner.

Towards these goals, we introduce VOYAGER, the first *LLM-powered embodied lifelong learning agent* to drive exploration, master a wide range of skills, and make new discoveries continually without human intervention in Minecraft. VOYAGER is made possible through three key modules (Fig. 2): 1) an **automatic curriculum** that maximizes exploration; 2) a **skill library** for storing and retrieving complex behaviors; and 3) a new **iterative prompting mechanism** that generates executable code for embodied control. We opt to use code as the action space instead of low-level motor commands because programs can naturally represent temporally extended and compositional actions [16, 22], which are essential for many long-horizon tasks in Minecraft. VOYAGER interacts with a blackbox LLM (GPT-4 [35]) through prompting and in-context learning [36–38]. Our approach bypasses the need for model parameter access and explicit gradient-based training or finetuning.

More specifically, VOYAGER attempts to solve progressively harder tasks proposed by the **automatic curriculum**, which takes into account the exploration progress and the agent’s state. The curriculum is generated by GPT-4 based on the overarching goal of “discovering as many diverse things as possible”. This approach can be perceived as an in-context form of *novelty search* [39, 40]. VOYAGER incrementally builds a **skill library** by storing the action programs that help solve a task successfully.

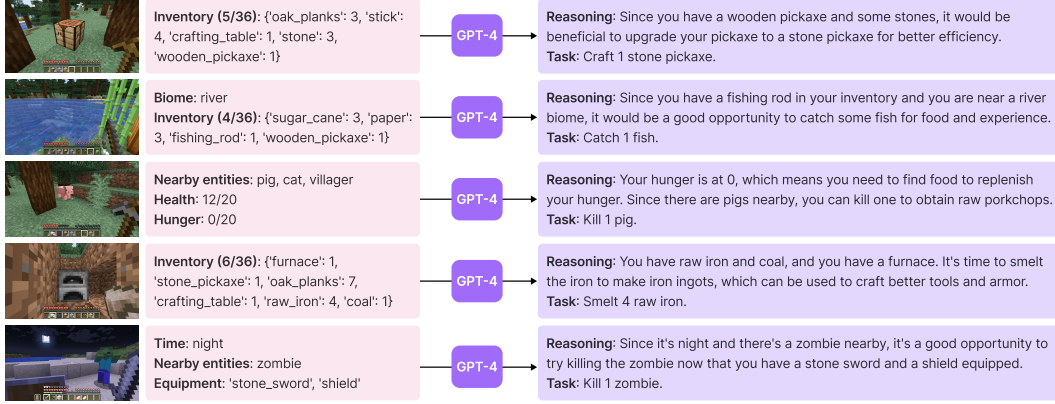


Figure 3: Tasks proposed by the automatic curriculum. We only display the partial prompt for brevity. See Appendix, Sec. A.3 for the full prompt structure.

Each program is indexed by the embedding of its description, which can be retrieved in similar situations in the future. Complex skills can be synthesized by *composing* simpler programs, which compounds VOYAGER’s capabilities rapidly over time and alleviates catastrophic forgetting in other continual learning methods [31, 32].

However, LLMs struggle to produce the correct action code consistently in one shot [41]. To address this challenge, we propose an **iterative prompting mechanism** that: (1) executes the generated program to obtain observations from the Minecraft simulation (such as inventory listing and nearby creatures) and error trace from the code interpreter (if any); (2) incorporates the feedback into GPT-4’s prompt for another round of code refinement; and (3) repeats the process until a self-verification module confirms the task completion, at which point we commit the program to the skill library (e.g., `craftStoneShovel()` and `combatZombieWithSword()`) and query the automatic curriculum for the next milestone (Fig. 2).

Empirically, VOYAGER demonstrates strong **in-context lifelong learning** capabilities. It can construct an ever-growing skill library of action programs that are reusable, interpretable, and generalizable to novel tasks. We evaluate VOYAGER systematically against other LLM-based agent techniques (e.g., ReAct [29], Reflexion [30], AutoGPT [28]) in MineDojo [23], an open-source Minecraft AI framework. VOYAGER outperforms prior SOTA by obtaining $3.3\times$ more unique items, unlocking key tech tree milestones up to $15.3\times$ faster, and traversing $2.3\times$ longer distances. We further demonstrate that VOYAGER is able to utilize the learned skill library in a new Minecraft world to solve novel tasks from scratch, while other methods struggle to generalize.

2 Method

VOYAGER consists of three novel components: (1) an automatic curriculum (Sec. 2.1) that suggests objectives for open-ended exploration, (2) a skill library (Sec. 2.2) for developing increasingly complex behaviors, and (3) an iterative prompting mechanism (Sec. 2.3) that generates executable code for embodied control. Full prompts are presented in Appendix, Sec. A.

2.1 Automatic Curriculum

Embodied agents encounter a variety of objectives with different complexity levels in open-ended environments. An automatic curriculum offers numerous benefits for open-ended exploration, ensuring a challenging but manageable learning process, fostering curiosity-driven intrinsic motivation for agents to learn and explore, and encouraging the development of general and flexible problem-solving strategies [42–44]. Our automatic curriculum capitalizes on the internet-scale knowledge contained within GPT-4 by prompting it to provide a steady stream of new tasks or challenges. The curriculum unfolds in a bottom-up fashion, allowing for considerable adaptability and responsiveness to the exploration progress and the agent’s current state (Fig. 3). As VOYAGER progresses to harder self-driven goals, it naturally learns a variety of skills, such as “mining a diamond”.

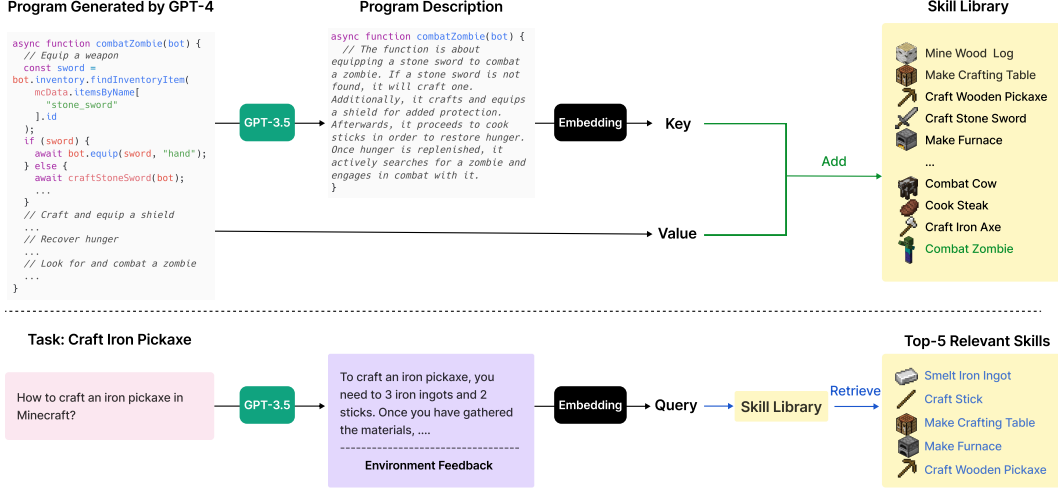


Figure 4: Skill library. **Top: Adding a new skill.** Each time GPT-4 generates and verifies a new skill, we add it to the skill library, represented by a vector database. The key is the embedding vector of the program description (generated by GPT-3.5), while the value is the program itself. **Bottom: Skill retrieval.** When faced with a new task proposed by the automatic curriculum, we first leverage GPT-3.5 to generate a general suggestion for solving the task, which is combined with environment feedback as the query context. Subsequently, we perform querying to identify the top-5 relevant skills.

The input prompt to GPT-4 consists of several components:

- (1) **Directives encouraging diverse behaviors and imposing constraints**, such as "My ultimate goal is to discover as many diverse things as possible ... The next task should not be too hard since I may not have the necessary resources or have learned enough skills to complete it yet.";
- (2) **The agent's current state**, including inventory, equipment, nearby blocks and entities, biome, time, health and hunger bars, and position;
- (3) **Previously completed and failed tasks**, reflecting the agent's current exploration progress and capabilities frontier;
- (4) **Additional context**: We also leverage GPT-3.5 to self-ask questions based on the agent's current state and exploration progress and self-answer questions. We opt to use GPT-3.5 instead of GPT-4 for standard NLP tasks due to budgetary considerations.

2.2 Skill Library

With the automatic curriculum consistently proposing increasingly complex tasks, it is essential to have a skill library that serves as a basis for learning and evolution. Inspired by the generality, interpretability, and universality of programs [45], we represent each skill with executable code that scaffolds temporally extended actions for completing a specific task proposed by the automatic curriculum.

The input prompt to GPT-4 consists of the following components:

- (1) **Guidelines for code generation**, such as "Your function will be reused for building more complex functions. Therefore, you should make it generic and reusable.";
- (2) **Control primitive APIs, and relevant skills** retrieved from the skill library, which are crucial for in-context learning [36–38] to work well;
- (3) **The generated code from the last round, environment feedback, execution errors, and critique**, based on which GPT-4 can self-improve (Sec. 2.3);
- (4) **The agent's current state**, including inventory, equipment, nearby blocks and entities, biome, time, health and hunger bars, and position;

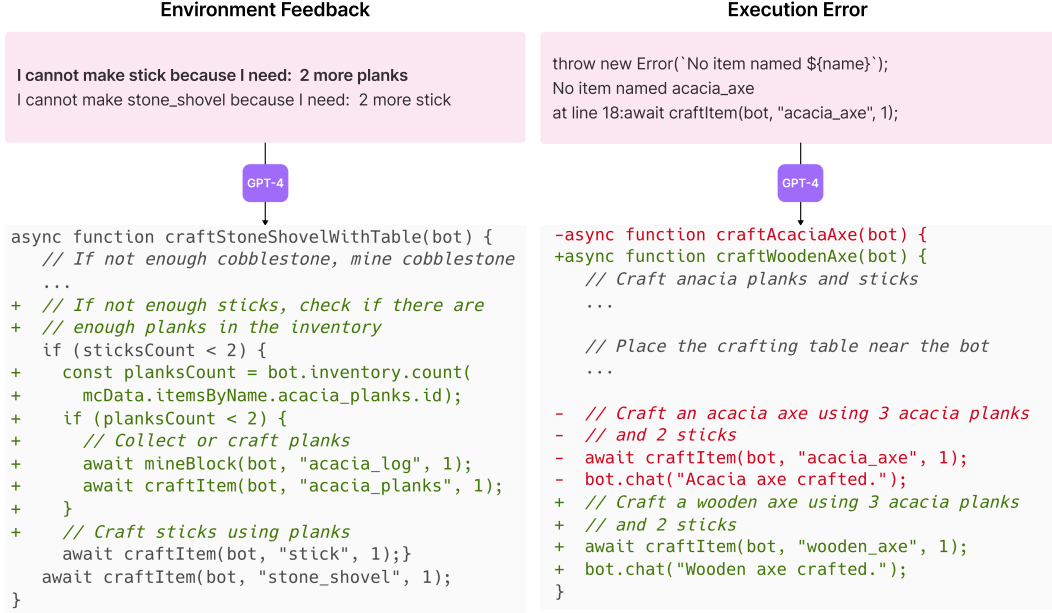


Figure 5: **Left: Environment feedback.** GPT-4 realizes it needs 2 more planks before crafting sticks. **Right: Execution error.** GPT-4 realizes it should craft a wooden axe instead of an acacia axe since there is no acacia axe in Minecraft. We only display the partial prompt for brevity. The full prompt structure for code generation is in Appendix, Sec. A.4.

(5) **Chain-of-thought prompting** [46] to do reasoning before code generation.

We iteratively refine the program through a novel iterative prompting mechanism (Sec. 2.3), incorporate it into the skill library as a new skill, and index it by the embedding of its description (Fig. 4, top). For skill retrieval, we query the skill library with the embedding of self-generated task plans and environment feedback (Fig. 4, bottom). By continuously expanding and refining the skill library, VOYAGER can learn, adapt, and excel in a wide spectrum of tasks, consistently pushing the boundaries of its capabilities in the open world.

2.3 Iterative Prompting Mechanism

We introduce an iterative prompting mechanism for self-improvement through three types of feedback:

- (1) **Environment feedback**, which illustrates the intermediate progress of program execution (Fig. 5, left). For example, “I cannot make an iron chestplate because I need: 7 more iron ingots” highlights the cause of failure in crafting an iron chestplate. We use `bot.chat()` inside control primitive APIs to generate environment feedback and prompt GPT-4 to use this function as well during code generation;
- (2) **Execution errors** from the program interpreter that reveal any invalid operations or syntax errors in programs, which are valuable for bug fixing (Fig. 5, right);
- (3) **Self-verification for checking task success.** Instead of manually coding success checkers for each new task proposed by the automatic curriculum, we instantiate another GPT-4 agent for self-verification. By providing VOYAGER’s current state and the task to GPT-4, we ask it to act as a critic [47–49] and inform us whether the program achieves the task. In addition, if the task fails, it provides a critique by suggesting how to complete the task (Fig. 6). Hence, our self-verification is more comprehensive than self-reflection [30] by both checking success and reflecting on mistakes.

During each round of code generation, we execute the generated program to obtain environment feedback and execution errors from the code interpreter, which are incorporated into GPT-4’s prompt for the next round of code refinement. This iterative process repeats until self-verification validates